- To present material that has been generated by AI as your own work or the work of someone else.

When submitting your assessment, you must:

- Check the box during the submission process that confirms you have adhered to the University's academic conduct policy and the expectations on use of GenAI in your assessment brief.

NOTICE: Your coding activity must all be completed on the GitHub repository. This logs your activity as you code, and the logs will be checked for submissions that have made use of AIgenerated code.

# 1 Requirements

The requirement for this project is to design and implement a web-based application that presents curated case studies of failures, limitations, or misuses of AI-supported environmental or geospatial systems, with the goal of supporting learning, reflection, and responsible practice.

Rather than showcasing successful AI systems, this project focuses on where, how, and why systems fail, and what can be learned from those failures.

## 1.1 Overview

AI systems are increasingly used to support decisions about the environment, land use, risk management, and sustainability. While many tools promise accuracy and efficiency, real-world deployments often encounter:

- incomplete or biased data,
- inappropriate assumptions,
- misinterpretation of outputs,
- over-confidence in automated results,
- lack of transparency or accountability.

These failures are rarely documented in accessible ways.

Your task is to build an "AI Failure Museum": a digital platform that presents structured, well-explained exhibits documenting AI-related failures or limitations, allowing users to explore what went wrong and why.

The emphasis is on explanation, governance, and learning, not blame.

### 1.2 Problem context and examples

Your system should support a range of failure types, for example:

- An AI system that produced misleading flood risk maps due to outdated data.
- A land-cover classification used beyond its spatial resolution.
- A decision dashboard that hid uncertainty, leading to over-confident conclusions.
- A system deployed in a context very different from the one it was trained for.
- A visualisation that looked authoritative but omitted key assumptions.

The cases used in your system may be fictionalised, simplified, or anonymised, but must be realistic and clearly explained.

### 1.3 The museum concept

Your system should support a range of failure types, for example:

- An AI system that produced misleading flood risk maps due to outdated data.
- A land-cover classification used beyond its spatial resolution.
- A decision dashboard that hid uncertainty, leading to over-confident conclusions.
- A system deployed in a context very different from the one it was trained for.
- A visualisation that looked authoritative but omitted key assumptions.

The cases used in your system may be fictionalised, simplified, or anonymised, but must be realistic and clearly explained.

## 2 Users and user profiles

Your application must support three user types, each with distinct needs:

#### 2.1 Visitors (Learners)

- Primary users.
- Browse exhibits and read case studies.
- View explanations and lessons learned.
- Complete short reflective quizzes.
- Optionally bookmark exhibits.

#### 2.2 Curators (Moderators)

- Create and edit exhibits.

- Upload supporting artefacts (maps, charts, screenshots).
- Annotate failures and lessons.
- Review visitor feedback or flags.
- View basic engagement analytics.

### 2.3 Developers (Maintainers)

- Access source code via GitHub.
- Run the system locally using documented setup steps.
- Extend the exhibit schema for new failure types.
- Deploy the system offline for demonstration.

# 3 Features

## 3.1 Core museum experience (required)

1. Browse or search a collection of AI failure exhibits.
2. View an exhibit page containing:
    - background context,
    - system purpose,
    - claimed outputs,
    - description of the failure,
    - contributing factors,
    - lessons learned.
3. View a timeline or sequence showing how the failure emerged.
4. Access supporting artefacts (e.g. screenshots, diagrams, summaries).
5. Navigate using a mobile-friendly, accessible interface.

## 3.2 Failure and exhibit model (required)

Your data model must support:

- Exhibits
    - title
    - domain (e.g. flooding, urban planning, biodiversity)
    - deployment context
    - intended use
- AI system description
    - system type (e.g. chatbot, classifier, dashboard, decision support)
    - inputs and assumptions
    - outputs presented to users
- Failure description
    - what went wrong
    - how it was detected
    - who was affected
- Contributing factors
    - data issues

    - design choices
    - organisational or governance issues

- Lessons learned

    - practical recommendations
    - warnings for future users

**3.3 Reflective interaction and learning (required)**

- Short quizzes linked to exhibits (e.g. “Which assumption caused the main issue?”).
- Reflection prompts encouraging users to think about:

    - uncertainty,
    - bias,
    - appropriate use.

- Points or badges for meaningful engagement (not simple page views).

Gamification must remain lightweight and appropriate for a university context.

**3.4 Curator tools (required)**

- Exhibit editor (create, update, archive).
- Artefact upload and management.
- Moderation queue for visitor flags or comments (optional but encouraged).
- Basic analytics:

    - most viewed exhibits,
    - quiz completion rates.

**3.5 Technical requirements (required)**

- Framework: Any framework, for example, Django.
- Authentication: role-based access control (visitor / curator / developer view).
- Online demo: system must work with locally seeded data.
- Security:
    - no secrets in Git,
    - environment variables documented via .env.example.
- Version control: GitHub repository with visible team contributions.

- Process: Kanban board showing task ownership and progress.

### 3.6 Privacy and responsible use (required)

- Publish a clear privacy policy.
- Minimise personal data collection.
- Use pseudonymous display names if required.
- Provide a basic “delete my data” mechanism.
- Avoid naming real organisations or individuals in a way that could cause harm.

## 4 Data pack

You will use a group-created dataset pack (CSV/JSON/SQLite) committed to the repository and shipped with the deployment artefact. You may optionally enrich data from open sources or APIs, but you must include a seeded dataset so the system works during marking even if external services are unavailable.

### 4.1 Minimum dataset contents (required)

- At least 20 exhibits.
- At least 20 supporting artefacts.
- At least 40 quizzes linked to exhibits.
- At least 5 failure categories.

The dataset must be sufficient to demonstrate all core features during marking without external dependencies.

## 5 Measures of success (evaluation)

You must define and report success measures. Suitable measures include:

- **Comprehension**: quiz performance after reading exhibits.
- **Reflection quality**: analysis of quiz explanations or responses.
- **Engagement**: revisit rates or time spent per exhibit.
- **Accessibility**: ability to use core features with assistive technologies.

CW1 must include at least one implemented measure with early results.

CW2 must include a fuller evaluation and discussion of limitations.

## 6 Process and deliverables

Your deliverables follow the module’s two-sprint structure. You must maintain a Kanban board, meeting records, and evidence of continuous version control.

### Intended Learning Outcomes assessed by the coursework

Coursework 1 and Coursework 2 assess all module Intended Learning Outcomes (ILOs).

- ILO1 – Function effectively as a member of a team.
- ILO2 – Apply an integrated or systems approach to the solution of complex problems.
- ILO3 – Apply knowledge of domain context, project and change management, and relevant legal matters including intellectual property rights.
- ILO4 – Select and apply appropriate materials, technologies, and processes, and recognise their limitations.
- ILO5 – Plan self-learning and development to support the activity of the wider team.
- ILO6 – Support an inclusive approach to teamwork and problem solving, recognising the responsibilities, benefits and importance of supporting equality, diversity, and inclusion.
- ILO7 – Evaluate the environmental and societal impact of solutions to complex problems and minimise adverse impacts.

### Where we expect to see evidence (indicative):

- ILO1: teamwork evidence in Kanban/process logs (CW1), handover and individual reflections (CW2).
- ILO2: architecture + data model + end-to-end prototype (CW1), final system integration and evaluation (CW2).
- ILO3: ethical/legal considerations, risk register, licence rationale (CW1), updated and consistent governance decisions (CW2).
- ILO4: justified technology choices, limitations and testing strategy (CW1), robustness, security baseline, and refined UX (CW2).
- ILO5: learning plan / skill acquisition in report (CW1), learning narrative and evidence in individual reflection (CW2).
- ILO6: inclusive team practices and accessibility considerations (CW1), accessibility checks and reflection on inclusion (CW2).
- ILO7: impact statement and mitigations (CW1), evaluation including societal/environmental impacts (CW2).

### 6.1 Sprint 1 (Coursework 1) – Prototype v1, report and demonstration

### Detailed deliverable instructions (what we expect you to produce and where it must appear in your ELE ZIP submission)

Your ELE submission ZIP is the source of truth for marking. GitHub is used to evidence professional practice (e.g., commits, issues, pull requests), and its URL must be included in 0_admin/submission.txt.

### Submission mechanism (Sprint 1)

Submission to ELE is a single ZIP file named GroupX_CW1.zip.

Notes: If you include credentials, use low-privilege demo accounts only and change passwords after marking.

### Prototype v0.1.0 (vertical slice) – minimum expected behaviour

6. A deployed online instance that a marker can open in a browser without special access (unless you have prior approval for restricted access).
7. End-to-end flow: scan/enter product ID → product passport page → complete one mission/quiz → see points/badge update.
8. At least one Game Keeper function demonstrated (e.g., create/edit a passport or publish a mission).
9. Basic error handling: invalid product IDs and missing evidence must produce user-friendly messages.
10. Basic security: role-based access control and no credentials committed to GitHub.

### Prototype Report v1 (group deliverable) – required sections and expectations

Provide a PDF report as 1_report/cw1_prototype_report.pdf in your ELE ZIP. Use headings that match the required sections below. You may include appendices in the PDF rather than splitting across multiple files.

- Executive summary (max 550 words): written for a decision maker; includes purpose, benefits, key risks, and what the prototype currently achieves.
- Project description and prioritised requirements (750–1,250 words): what problem you solve, scope boundaries, and success criteria. Requirements must be captured as GitHub issues (see below).
- Architecture (v1): a high-level component diagram and explanation of each component; include data storage, external services, and deployment topology. Store at the relevant folder in your ELE ZIP (figures in the same folder).
- Data model and dataset description: the passport schema (products, nodes, stages, evidence) and how your seeded dataset is structured and generated/curated.
- Evaluation plan (initial): at least one implemented measure with early results (e.g., comprehension quiz accuracy from pilot users). Clearly state what you will evaluate more thoroughly in CW2.
- Risk register: technical risks, delivery risks, and operational risks; include mitigations and owners. Store at the relevant folder in your ELE ZIP and link it from the report.

### Ethical and legal considerations (required)

Provide a PDF as 3_ethics_and_licensing/ethical_and_legal_considerations.pdf in your ELE ZIP. This must be detailed enough for a reader to decide whether a formal ethics approval process would be required. At minimum, address: (1) privacy/GDPR and data minimisation, (2) acceptable use and safety of gamification, (3) risks of misinformation or misleading claims, (4) accessibility and inclusion, and (5) security controls (authentication, authorisation, storage, logging). Include a short conclusion stating whether ethics approval is needed and why.

Submission to ELE is a single ZIP file named GroupX_CW2.zip.

Individual reflection (submitted separately by each student on ELE):

reflection.pdf (800–1,000 words; includes evidence links: PRs/issues/commits; and an AI-Minimal compliance statement)

## Live demo and presentation (10 minutes) – what to cover

11. **30 seconds**: briefly introduce the problem context and explain what users gain from the AI Failure Museum (i.e. understanding how and why AI-supported decisions can fail, and how to recognise such failures in practice).
12. **6–7 minutes**: demonstrate the vertical slice of the system:

- browse or search the museum exhibits,
- open a single exhibit and walk through:
    - the decision context,
    - what the AI system claimed or supported,
    - what went wrong and contributing factors,
    - lessons learned,
- complete a short reflective quiz linked to the exhibit,
- show how points, badges, or feedback are updated,
- demonstrate **one Curator action** (e.g. creating, editing, or annotating an exhibit).

13. **1–2 minutes**: show the GitHub repository structure, highlighting:

- where key documentation is located,
- how to run the system locally,
- how tests are executed (if applicable),
- key issues or milestones tracked during development.

14. **Final minute**: outline what is planned for Sprint 2, including:

- additional exhibit types or features,
- improvements to accessibility or usability,
- the main technical or organisational risks being managed.

Provide your slides as 5_presentation/cw1_demo_slides.pdf in your ELE ZIP.

| **Deliverable** | **What it must show (minimum)** |
|---|---|
| **Prototype v0.1.0 (GitHub release)** | End-to-end vertical slice: browse → exhibit view → reflective quiz → feedback/points update. |

| | |
|---|---|
| **Prototype Report v1** | Requirements v1; architecture and data model; early evaluation evidence; risk register; privacy policy; deployment steps. |
| **Demo (live)** | 10-minute walkthrough of the vertical slice, followed by 1–2 minutes of questions as scheduled by the module team. |

### 6.2 Sprint 2 (Coursework 2) – Final prototype, client handover and presentation

### Detailed deliverable instructions (what we expect you to produce and where it must appear in your ELE ZIP submission)

### Submission mechanism (Sprint 2)

- Create a Git tag named cw2_final_v1 that marks the final repository state submitted for Coursework 2.
- Create a GitHub Release from this tag, including release notes describing what changed since Sprint 1.
- Submit submission.txt on ELE containing: repository URL, Release URL for cw2_final_v1, deployed demo URL, and (if applicable) links to any large assets (datasets/models) stored outside Git.

### Final prototype v1.0.0 – minimum expected completeness

- All core features complete and stable (single scan passport, evidence scoring, missions/quizzes, role-based admin tools).
- Expanded dataset coverage and improved UX (mobile-first, accessible, clear error messages).
- Testing evidence: unit tests and/or documented manual integration tests, and a clear ‘how to run tests’ section.
- Security baseline: secrets management, least-privilege roles, and a short security checklist (OWASP-style) recorded in docs/security.md.

### Client handover documentation (group deliverable) – required sections

Your CW2 documentation should enable a client to install, operate, maintain, and extend your prototype. Provide the handover pack as PDFs inside 2_handover_pack/ in your ELE ZIP. (You may also keep a well-structured docs/ folder in GitHub for professionalism.) You may split the handover into multiple Markdown files; however, docs/handover/README.md must act as the index.

- Updated README front page: purpose, developers' names, licence, dependency overview, and links to all key documentation.
- Architecture (final): updated component diagram and deployment view, reflecting the final prototype (docs/architecture/README.md).
- Development environment: 'change and run' instructions for developers (docs/handover/development_setup.md).
- Repository layout and tests/CI: what each top-level folder does; how to run tests; and how to validate a build (docs/handover/repo_and_tests.md).
- System requirements and deployment: required services, environment variables, and stepby-step deployment instructions (docs/handover/deployment.md).
- Operations and maintenance: routine admin tasks (user management, backups), logging/monitoring, and common troubleshooting steps (docs/handover/operations_and_maintenance.md).
- Data management: seeded dataset description, how to import/update passports, and any data governance decisions (docs/data/README.md).
- Updated software/data inventory (software_bom.xlsx or inventory.xlsx) and updated licence decision rationale (docs/decisions/licence-decision.md).

### Final report (group) – required content

- Evaluation: report outcomes for your success measures (comprehension, transparency, engagement, accessibility). Explain method, participants (if any), and limitations/threats to validity.
- Critical reflection on design trade-offs: what you chose not to do and why (scope management).
- Ethical and legal update: confirm that docs/ethical_and_legal_considerations.md reflects the final system, especially data handling and user safety.
- Future roadmap: 3–6 clearly scoped improvements, with rationale and estimated effort.

Provide the final report as 1_report/cw2_final_report.pdf in your ELE ZIP.

### Individual reflection (Coursework 2 – individual deliverable)

Each student must submit an individual reflection on ELE (not in the group repository). Suggested length: 800–1,000 words. This reflection is used to evidence individual learning and contribution and may be used to resolve contribution disputes.

- Your role and contributions: describe what you owned (features, testing, documentation, deployment). Reference concrete evidence (PR links, issue IDs, commits).

- What you learned: at least three specific technical or professional learning points linked to module outcomes (e.g., requirements negotiation, risk management, testing, deployment).
- Challenges and how you addressed them: one technical challenge and one teamwork/process challenge; what changed as a result.
- Responsible computing: what ethical/legal risk you personally focused on (privacy, accessibility, safety) and how you mitigated it.
- AI-Minimal compliance statement: confirm you adhered to the brief and did not use GenAI for code generation or content generation beyond spelling/grammar checks.

**Live demo and presentation (10 minutes) – what to cover**

- 1 minute: recap problem and what is now delivered.
- 6–7 minutes: demo the final system including one admin workflow, plus a brief look at analytics/dashboard (if implemented).
- 1–2 minutes: show deployment and handover docs (where to find install/run/maintain instructions).
- Final minute: evaluation highlights, limitations, and next steps.

| Deliverable | What it must show (minimum) |
|---|---|
| Final Release v1.0.0 (GitHub release) | Robust, complete core features; expanded tests; refined UX; deployment verified. |
| Client Handover Pack | How to use; how to deploy; how to update passports/missions; maintenance and roadmap. |
| Final Report + Presentation | Substantive evaluation; limitations/threats; responsible |
| (live) | computing; clear demo narrative and Q&A. |

# 7 Safety and responsible use of campus context

- All missions and interactions must be safe and appropriate to a University client.
- Do not create challenges that encourage unsafe behaviour, trespassing, or harassment.
- Any location-based verification (QR codes) must be used responsibly and must not cause disruption.

# 8 Summary

This project challenges you to design a thoughtful, well-governed software system that supports learning from AI failures in environmental contexts. Success will be judged not only on technical correctness, but on clarity, responsibility, and the ability to communicate complex ideas accessibly.

# [END OF SPECIFICATION]

Document owner: Module team (COMM2020)

Date: 25 December 2025